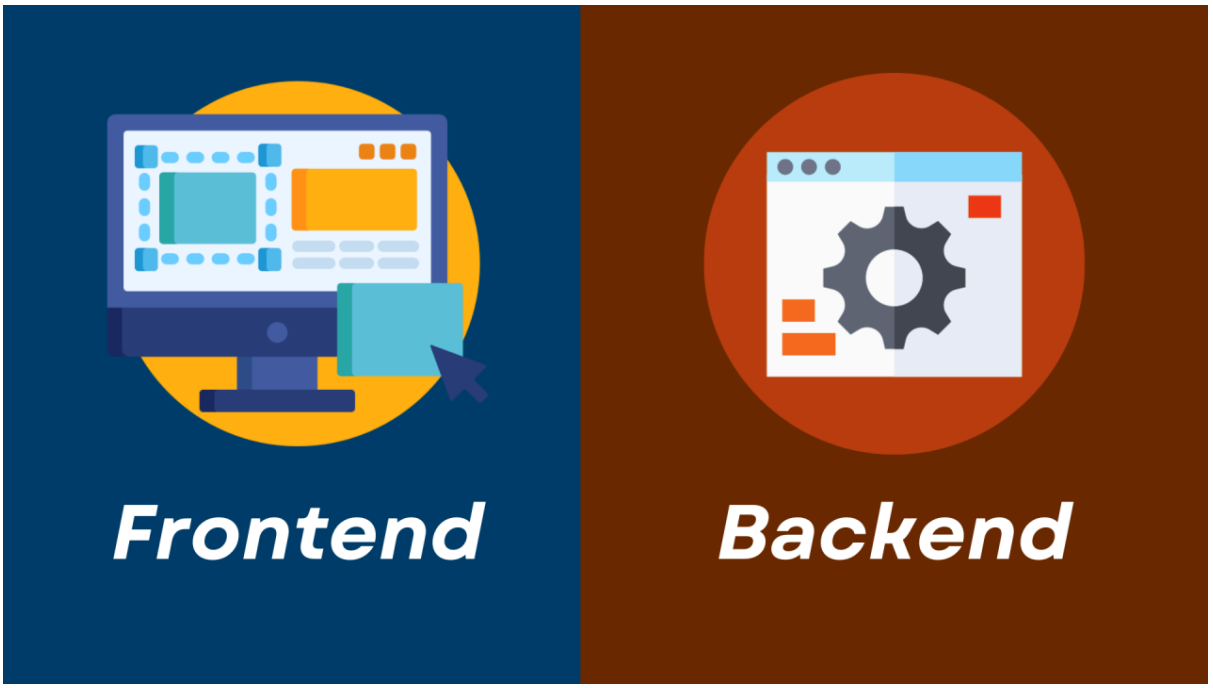
OP1-B1-NODE-03



Esteve Amargant Blanco
Maria Pilar Moreno
Frameworks de forntend i backends

Lliurament pdf amb captures de les respostes de postman, i codi de server.js i package.json degudament comentat.

📋 Enunciat: Ampliació del Servidor Node.js

🎯 Objectiu

Ampliar el servidor Node.js creat anteriorment afegint noves rutes amb funcionalitats específiques i validacions.

📝 Tasques a implementar

Tasca 1: Ruta de Data i Hora

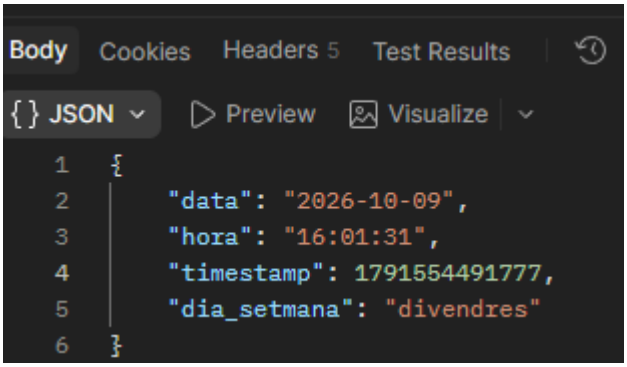
Ruta: `GET /data`

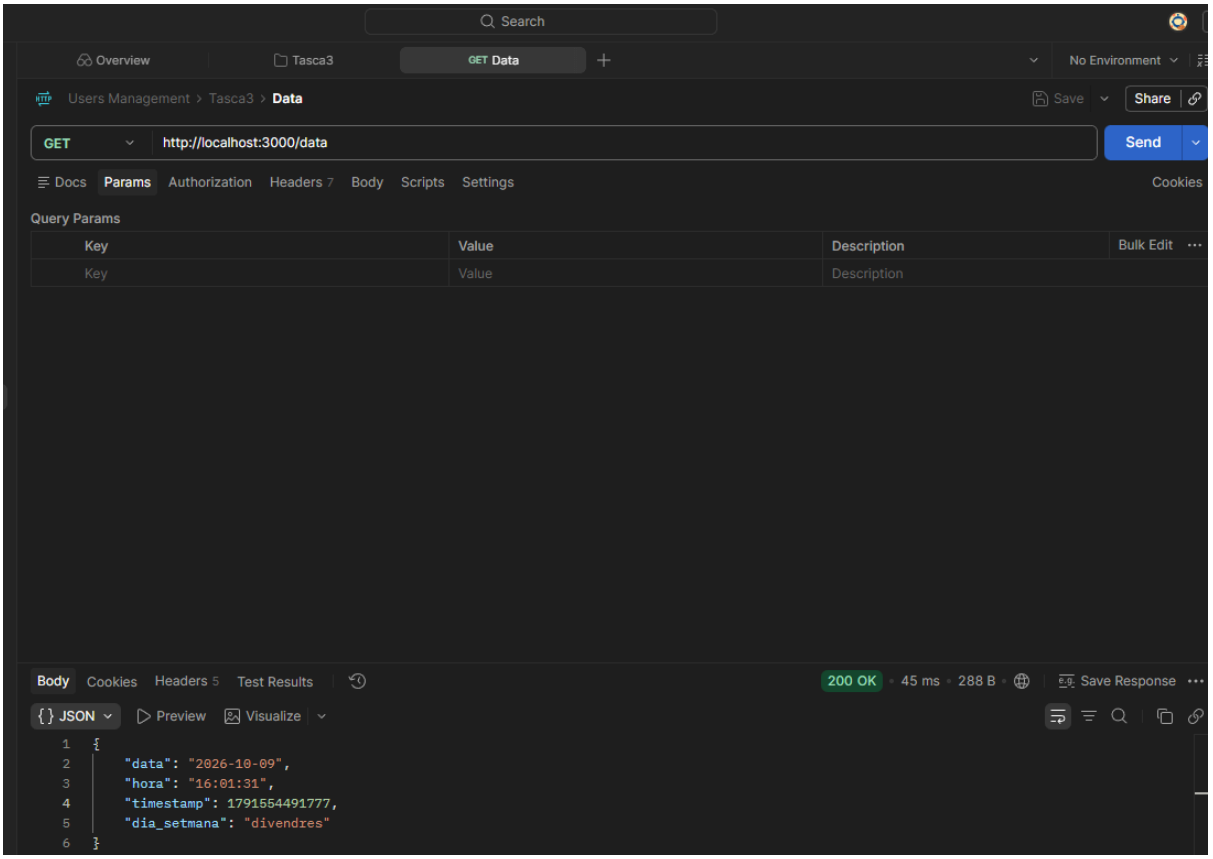
Descripció: Crear una ruta que retorni la data i hora actuals en format JSON.
Resposta esperada:

```
{
  "data": "2026-10-08",
  "hora": "19:55:30",
  "timestamp": 1728408930000,
  "dia_setmana": "dimarts"
}
```

💡 **Pista**: Utilitza l'objecte Date de JavaScript. Pots usar new Date() i els seus mètodes com .toLocaleDateString(), .toLocaleTimeString(), etc.

Dins del nostre server.js , hem de afegir el if de que sigui el metode GET i la ruta /data i posar el codi per treure les daddes i enviarles en format JSON amb el metode enviarJSON (request, status (de resposta , 200, 404,403…) i les dades de JSON).
Posme el servidor a escoltar amb npm run dev (el dev fa un –watch sobre el server i no fa falta anra parant i tornant a engegar per fer canvis) i executem la ruta sobre el Postman:






```
if (metode === "GET" && url.pathname === "/data") {
  const ara = new Date();

  const dies = [
    "diumenge",
    "dilluns",
    "dimarts",
    "dimecres",
    "dijous",
    "divendres",
    "dissabte",
  ];

  return enviarJSON(res, 200, {
    data: ara.toLocaleDateString("sv-SE"), //AAAA-MM-DD
    hora: ara.toLocaleTimeString("ca-ES", { hour12: false }), //HH:MM:SS
    timestamp: ara.getTime(),
    dia_setmana: dies[ara.getDay()], // getDay() retorna un número: 0 = di
  });
}
```

Tasca 2: Ruta d'Usuari amb Validació

Ruta: `POST /usuari`

Descripció: Crear una ruta que rebi dades d'usuari i les validi abans de processar-les.

Dades d'entrada esperades:

```
{
  "nom": "Joan Garcia",
  "email": "joan@exemple.com"
}
```

Validacions requerides:

- Nom: Mínim 2 caràcters, màxim 50, només lletres i espais sense accents.
- Email: Format vàlid d'email (contenir @ i punt)

Resposta d'èxit:

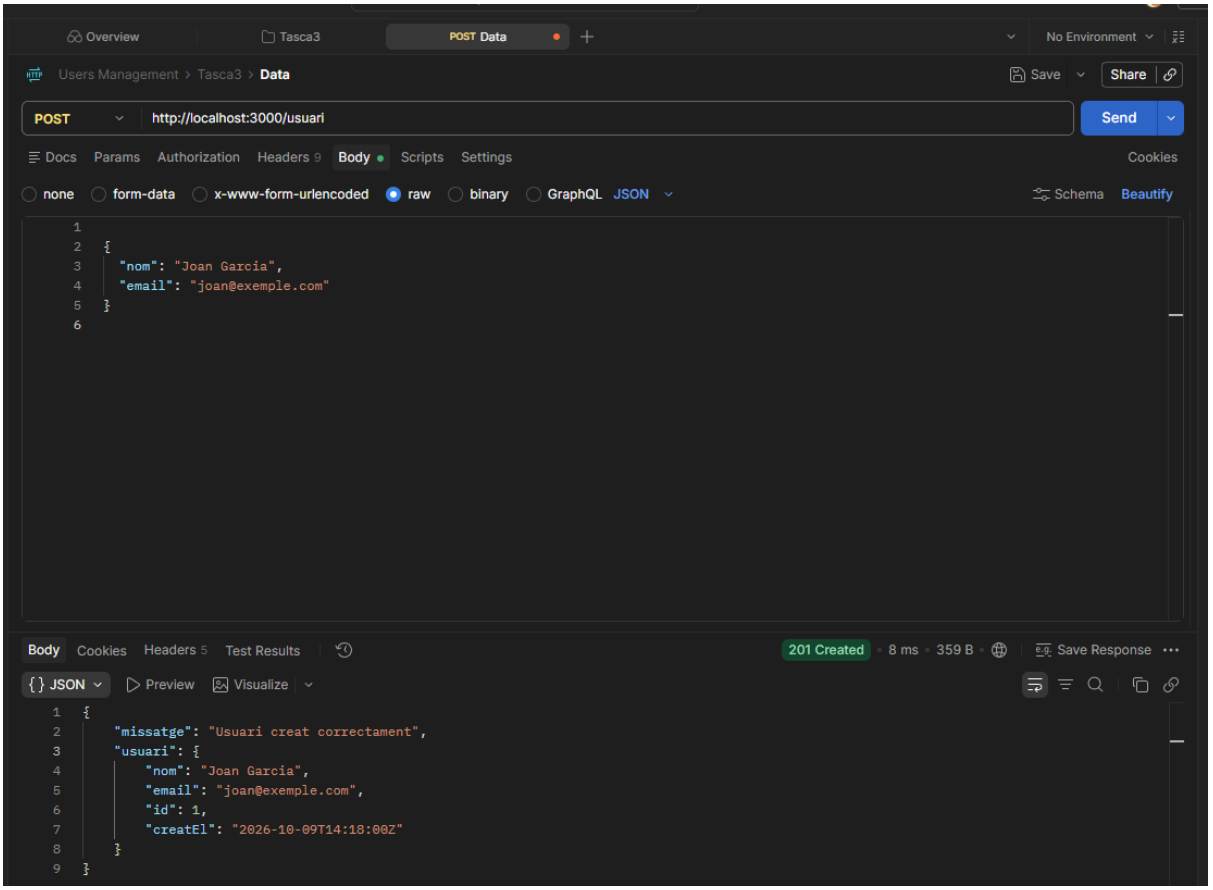
```
{
  "missatge": "Usuari creat correctament",
  "usuari": {
    "nom": "Joan Garcia",
    "email": "joan@exemple.com",
    "id": 1,
    "creatEl": "2026-10-01T19:55:30Z"
  }
}
```

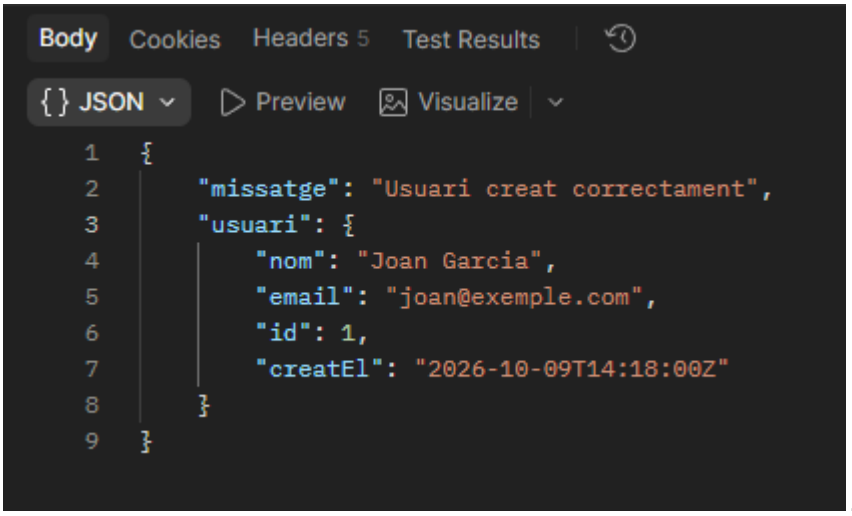
Resposta d'error:

```
{
  "error": "Dades invàlides",
  "data": ["El nom ha de tenir entre 2 i 50 caràcters", "Format d'email invàlid"]
}
```

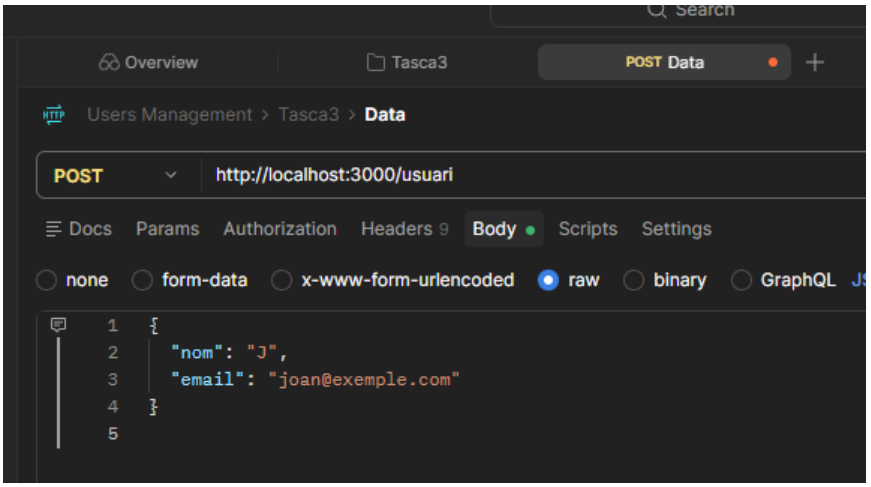
Igual que abans, hem de posar el if que coincideixi amb la ruta i el metode utilitzat (POST) i a dins posem tot el que volem que faci, primer recullim les dades de el body amb `req.on("data", () => {` i un cop ha acabt executem `req.on("end", () => {` que nomes s'hi entra quan s'ha rebut tot el bosy, per contrapart per sortir d'aqui hem d'utilitzar un return si no es seguiria executant el que hi hauria a sota. Un cop tenim les dades de l body, les "parsejem" i si pasen totes les validacions les enviem via enviarJson amb un camp missatge. Realment hi ha mes codi de validacio i control d'errors que no cap altre cosa. Per saber quina validacio ha fallat i quin missage d'error enviar, ho fem amb un array i al if de la validacio que hi falli li fem push del error, quan arriba al final abans de enviarJSON amb les dades, si el array d'errors es mes gran que 0 s'envia el enviarJson amb el error del array.

Cas correcte:

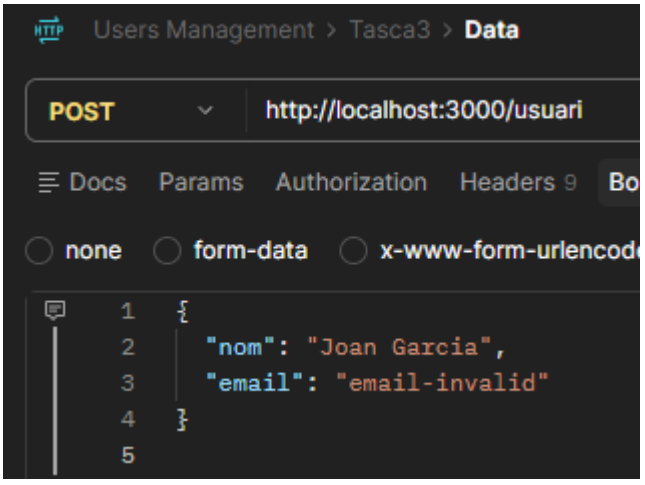


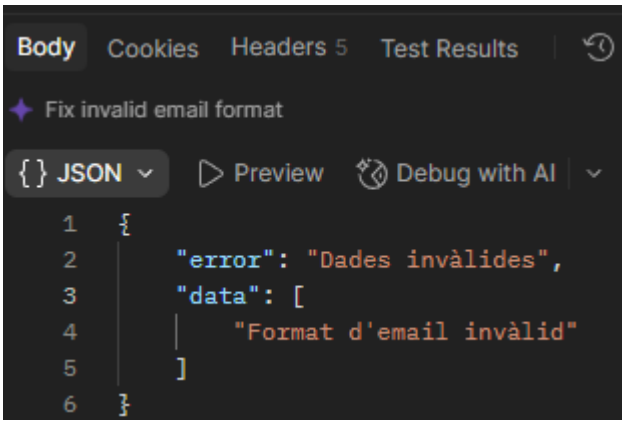


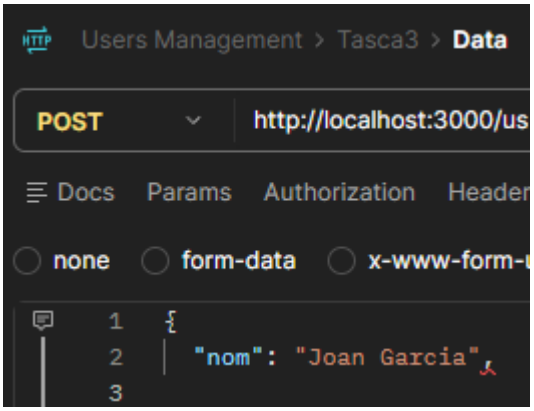
Cas incorrecte:

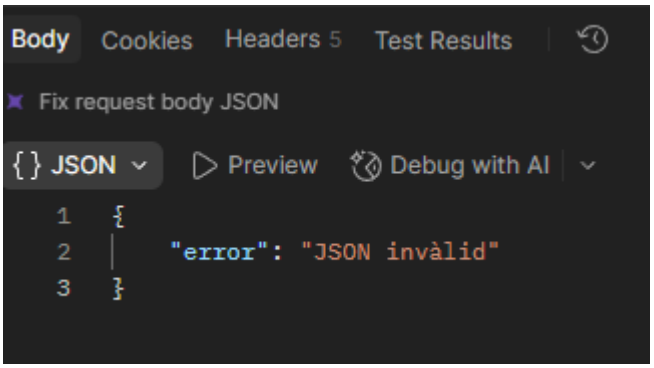









Users Management > Tasca3 > Data
POST
http://localhost:3000/us
Docs
Params
Authorization
Header
none
form-data
x-www-form-
1 {
2 "nom": "Joan Garcia",
3


Body
Cookies
Headers 5
Test Results
Fix request body JSON
{ } JSON
Preview
Debug with AI
1 {
2 "error": "JSON invàlid"
3 }

Tasca 3: Ruta d'Informació del Sistema

Ruta: `GET /api/info`

Descripció: Crear una ruta que retorni informació sobre el servidor i el sistema.

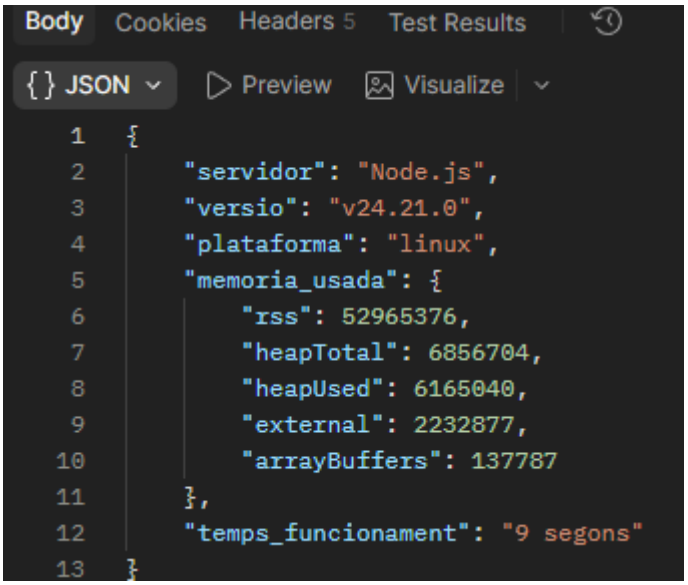
Resposta esperada:

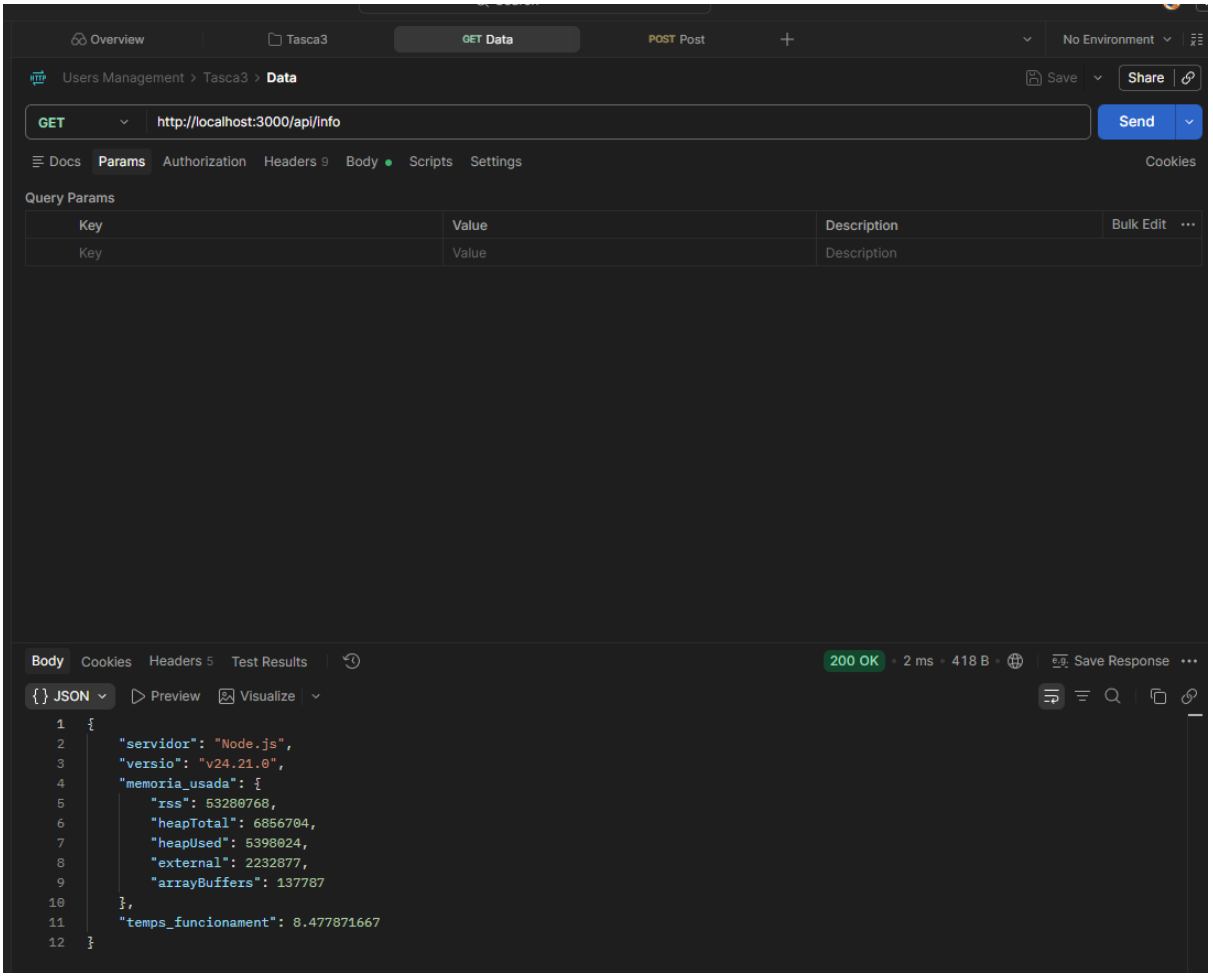
```
{
  "servidor": "Node.js",
  "versio": "v24.21.0",
  "plataforma": "linux",
  "memoria_usada": "25.6 MB",
  "temps_funcionament": "120 segons"
}
```

💡 Pista: Utilitza l'objecte global process de Node.js. Mira process.version, process.platform, process.memoryUsage() i process.uptime().

Exactamen igual que amb les altres rute, comprobem el metode i la url i via enviarJson enviem les dades de process.

```
}
if (metode === "GET" && url.pathname === "/api/info") {
  return enviarJSON(res, 200, {
    servidor: "Node.js",
    versio: process.version,
    plataforma: process.plataform,
    memoria_usada: process.memoryUsage(),
    temps_funcionament: process.uptime(),
  });
}
```




Overview
Tasca3
GET Data
POST Post
No Environment
Users Management > Tasca3 > Data
Save
Share
GET
http://localhost:3000/api/info
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers 5
Test Results
200 OK
2 ms
418 B
Save Response
JSON
Preview
Visualize
{
"servidor": "Node.js",
"versio": "v24.21.0",
"memoria_usada": {
"rss": 53280768,
"heapTotal": 6856704,
"heapUsed": 5398024,
"external": 2232877,
"arrayBuffers": 137787
},
"temps_funcionament": 8.477871667
}

Tasca 4: Ruta de Calculadora

Ruta: `GET /api/calculadora/:operacio/:num1/:num2`

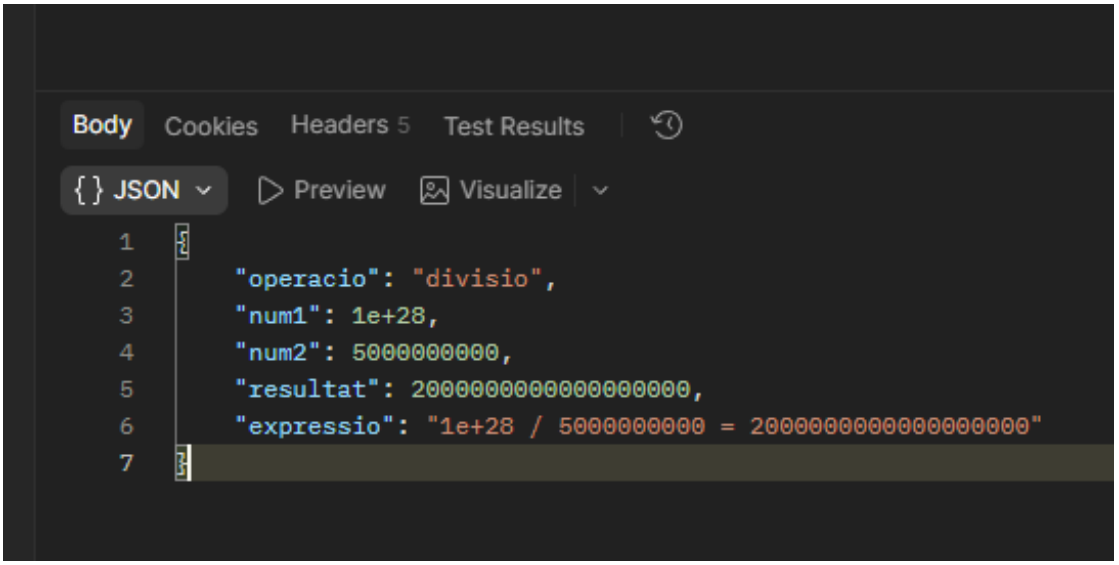
Descripció: Crear una ruta dinàmica que faci operacions matemàtiques bàsiques.
Operacions suportades: suma, resta, multiplicacio, divisio
Exemple d'ús: GET /api/calculadora/suma/5/3
Resposta esperada:

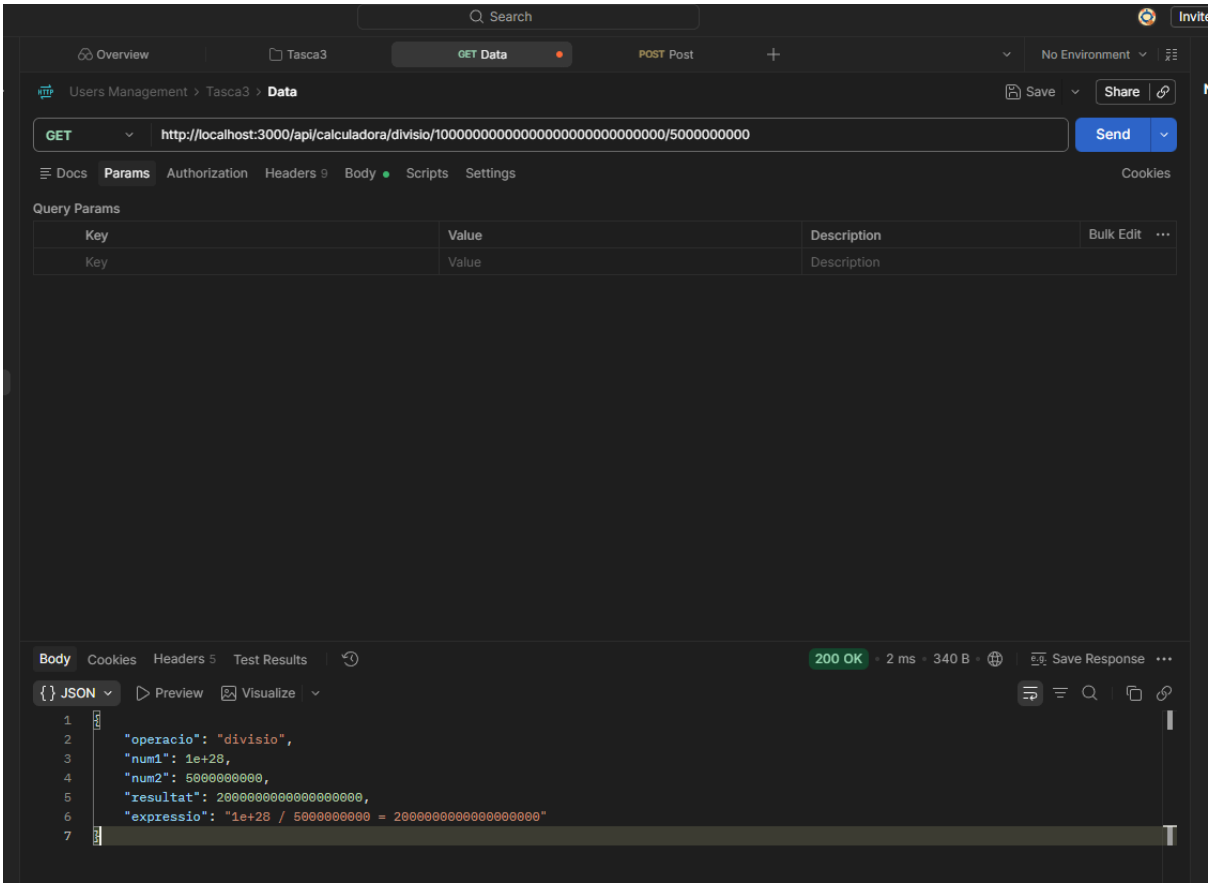

```
{
  "operacio": "suma",
  "num1": 5,
  "num2": 3,
  "resultat": 8,
  "expressio": "5 + 3 = 8"
}
```

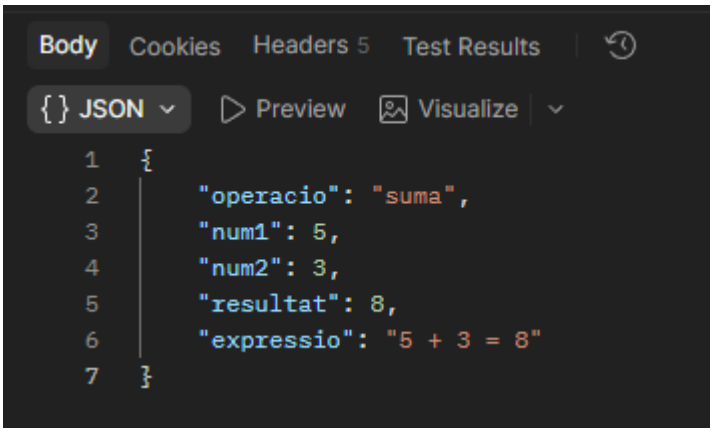

💡 Pistes:

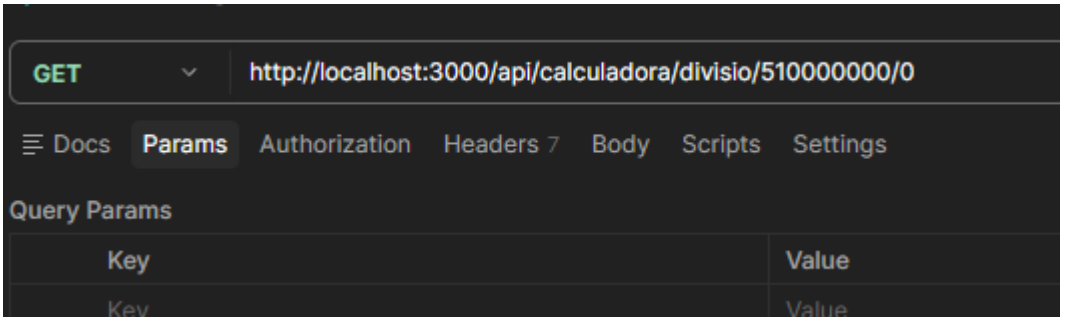
- Utilitza url.pathname.split('/') per extreure els paràmetres. Pista: aquesta ruta no es pot comparar amb ===, perquè canvia cada vegada. Mira startsWith().
- Converteix els strings a números amb parseInt() o parseFloat()
- Afegeix validació per evitar divisió per zero

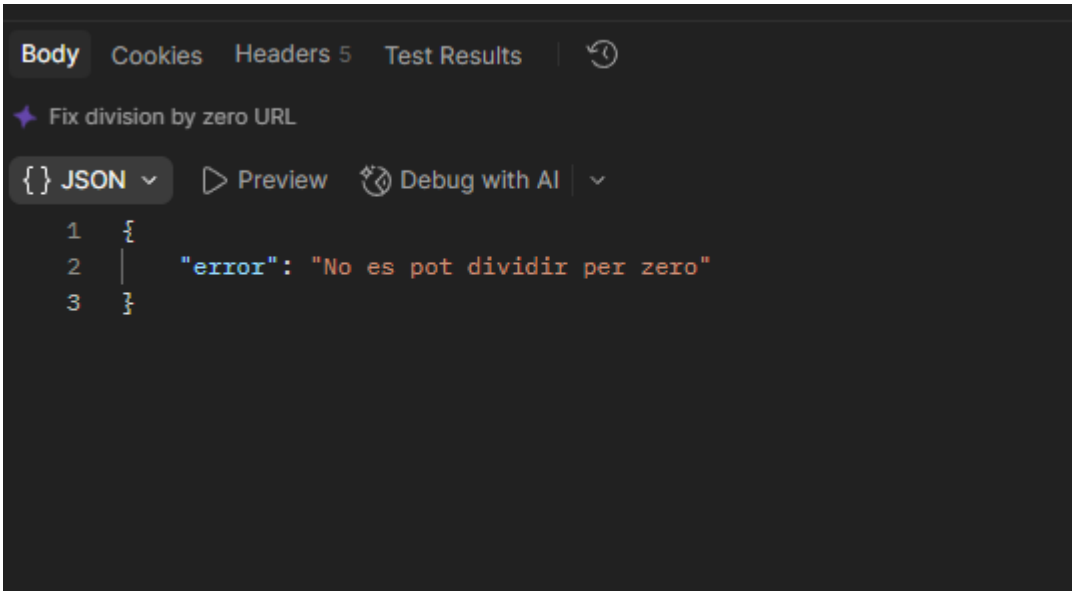
En aquest cas al agafar les dades que volem desde la url i fer casos depenen de la operacio que posin, per entrar en aquest if hem de posar la ruta comuna, en aquest cas /api/calculadora. Un cop a dins, com que sabem que el split de la url sempre donara 6, fem la comprobacio i si no hi ha 6 exactament, enviem error 400. Quan ja ho tenim separat per split en un array, comprovem que els numeros siguin numeros (previament fet el parse) i amb un switch posem els casos d'us de les diferents operacions.




Search
Overview
Tasca3
GET Data
POST Post
No Environment
Users Management > Tasca3 > Data
Save
Share
GET
http://localhost:3000/api/calculadora/divisio/10000000000000000000000000000/5000000000
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers 5
Test Results
200 OK
2 ms
340 B
Save Response
JSON
Preview
Visualize
{
"operacio": "divisio",
"num1": 1e+28,
"num2": 5000000000,
"resultat": 2000000000000000000,
"expressio": "1e+28 / 5000000000 = 2000000000000000000"
}


Body
Cookies
Headers 5
Test Results
JSON
Preview
Visualize
{
"operacio": "suma",
"num1": 5,
"num2": 3,
"resultat": 8,
"expressio": "5 + 3 = 8"
}


GET
http://localhost:3000/api/calculadora/divisio/510000000/0
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Query Params
Key
Value


Body
Cookies
Headers 5
Test Results
Fix division by zero URL
JSON
Preview
Debug with AI
1 {
2 "error": "No es pot dividir per zero"
3 }